

PYTHON CLINICAL DATA ANALYSIS

CODE SAMPLE

Data Cleaning • Biostatistics • Automated Excel & Word Reporting

Selected sections from a reusable Python workflow developed for an anonymized 20-participant clinical pilot study.

PROJECT SCOPE

- Multi-sheet Excel data cleaning and standardization
- Missing-data validation and quality checks
- Mann–Whitney U, Fisher’s exact, and Spearman analyses
- Automated Excel outputs and Word report generation

Portfolio sample only. No patient-identifying data or source dataset is included.

1. PROJECT SETUP & CONFIGURATION

This section defines the anonymized input file, reusable output paths, sample size, statistical thresholds, and analysis parameters.

```
from pathlib import Path

KLASOR_YOLU = Path(".")

EXCEL_DOSYA_ADI = "anonymized_clinical_data.xlsx"
EXCEL_YOLU = KLASOR_YOLU / EXCEL_DOSYA_ADI

CIKTI_EXCEL_ADI = "clinical_analysis_outputs.xlsx"
CIKTI_WORD_ADI = "clinical_analysis_report.docx"

CIKTI_EXCEL_YOLU = KLASOR_YOLU / CIKTI_EXCEL_ADI
CIKTI_WORD_YOLU = KLASOR_YOLU / CIKTI_WORD_ADI

MIN_HASTA_NO = 1
MAX_HASTA_NO = 20
HASTA_GRUBU_SON_NO = 10

VHIT_DUSUK_GAIN_ESIGI = 0.80
OPK_DUSUK_ESIGI = 56
SAKKAD_UZUN_LATANS_ESIGI = 250
ANLAMLILIK_DUZEYI = 0.05
```

2. DATA CLEANING & STANDARDIZATION

This section standardizes inconsistent Excel headers, converts mixed-format values into numeric data, removes identifying fields, and preserves missing values correctly.

```
def metin_norm(deger):
    """Metinleri güvenli sütun araması için standartlaştırır."""
    if pd.isna(deger):
        return ""

    metin = str(deger).strip().lower()
    ceviri = str.maketrans({
        "ç": "c", "ğ": "g", "ı": "i", "İ": "i",
        "ö": "o", "ş": "s", "ü": "u",
    })

    metin = metin.translate(ceviri)
    metin = unicodedata.normalize("NFKD", metin)
    metin = "".join(
        karakter for karakter in metin
        if not unicodedata.combining(karakter)
    )
    metin = re.sub(r"^[a-z0-9%]+", " ", metin)

    return re.sub(r"\s+", " ", metin).strip()

def sayisal_deger(deger):
    """Karışık biçimli değerleri sayısal veriye dönüştürür."""
    if pd.isna(deger):
        return np.nan

    if isinstance(
        deger,
        (int, float, np.integer, np.floating),
    ):
        return float(deger)

    metin = str(deger).strip().lower()

    if metin in {"", "nan", "none", "-", "- "}:
        return np.nan

    metin = metin.replace("%", "").replace(", ", ".")
    eslesme = re.search(r"[-+]?[d+](?:\.\d+)?", metin)

    return float(eslesme.group()) if eslesme else np.nan
```

2.1 PRIVACY-SAFE EXCEL LOADING

The workflow detects the correct header row, removes personal name fields, validates participant numbers, and prevents duplicate records during data loading.

```
def tabloyu_guvenli_oku(excel_yolu, sayfa_adi):
    """Excel sayfasını güvenli biçimde okur ve anonimleştirir."""
    ham = pd.read_excel(
        excel_yolu,
        sheet_name=sayfa_adi,
        header=None,
        dtype=object,
    )

    ham = (
        ham.dropna(axis=0, how="all")
        .dropna(axis=1, how="all")
        .reset_index(drop=True)
    )

    baslik_satiri = None

    for i in range(min(20, len(ham))):
        satir = [metin_norm(x) for x in ham.iloc[i]]

        if any(
            "hasta" in hucre and "no" in hucre
            for hucre in satir
        ):
            baslik_satiri = i
            break

    if baslik_satiri is None:
        raise ValueError(
            f"{sayfa_adi}: participant number header not found."
        )

    basliklar = benzersiz_sutun_adlari(
        ham.iloc[baslik_satiri].tolist()
    )

    df = ham.iloc[baslik_satiri + 1:].copy()
    df.columns = basliklar
    df = df.dropna(axis=0, how="all")

    hasta_no_sutunu = next(
        (
            sutun for sutun in df.columns
            if "hasta" in metin_norm(sutun)
            and "no" in metin_norm(sutun)
        ),
        None,
    )

    if hasta_no_sutunu is None:
        raise ValueError("Participant number column not found.")

    df.rename(
        columns={hasta_no_sutunu: "Hasta No"},
        inplace=True,
    )

    df["Hasta No"] = df["Hasta No"].apply(
        hasta_no_sayisal
    )
```

2.1 PRIVACY-SAFE EXCEL LOADING — CONTINUED

```
df = df[df["Hasta No"].between(
    MIN_HASTA_NO,
    MAX_HASTA_NO,
)].copy()

kimlik_sutunlari = [
    sutun for sutun in df.columns
    if metin_norm(sutun) in {
        "isim",
        "isim soyisim",
        "ad",
        "ad soyad",
        "hasta adi",
    }
]

df.drop(
    columns=kimlik_sutunlari,
    inplace=True,
    errors="ignore",
)

if df["Hasta No"].duplicated().any():
    df = (
        df.groupby("Hasta No", as_index=False)
        .agg(ilk_dolu_deger)
    )

df["Kisi_ID"] = df["Hasta No"].apply(kisi_id_uret)
df["Grup"] = df["Hasta No"].apply(grup_uret)

return df.sort_values("Hasta No").reset_index(drop=True)
```

3. STATISTICAL ANALYSIS

This section applies non-parametric statistical methods to compare patient and control groups, evaluate categorical findings, and examine correlations between clinical variables.

3.1 MANN-WHITNEY U TEST

The Mann-Whitney U test compares continuous clinical variables between patient and control groups without assuming a normal distribution.

```
istatistik_sonuclar = []

for degisken in analiz_degiskenleri:
    hasta = hipotez_tablo.loc[
        hipotez_tablo["Grup"] == "Hasta",
        degisken,
    ].dropna()

    kontrol = hipotez_tablo.loc[
        hipotez_tablo["Grup"] == "Kontrol",
        degisken,
    ].dropna()

    if len(hasta) > 0 and len(kontrol) > 0:
        test = mannwhitneyu(
            hasta,
            kontrol,
            alternative="two-sided",
        )

        u_degeri = float(test.statistic)
        p_degeri = float(test.pvalue)

        yorum = (
            "Significant difference"
            if p_degeri < ANLAMLILIK_DUZEYI
            else "No significant difference"
        )
    else:
        u_degeri = np.nan
        p_degeri = np.nan
        yorum = "Test could not be calculated"

    istatistik_sonuclar.append({
        "Variable": degisken,
        "Patient_n": len(hasta),
        "Control_n": len(kontrol),
        "Patient_Median": hasta.median()
        if len(hasta) else np.nan,
        "Control_Median": kontrol.median()
        if len(kontrol) else np.nan,
        "U_Statistic": u_degeri,
        "p_Value": p_degeri,
        "Interpretation": yorum,
    })

istatistik_tablo = pd.DataFrame(istatistik_sonuclar)
```

3.2 FISHER'S EXACT TEST

Fisher's exact test evaluates associations between patient groups and categorical clinical findings when sample sizes are small.

```
kategorik_sonuclar = []

for degisken in kategorik_degiskenler:
    veri = hipotez_tablo[
        ["Grup", degisken]
    ].dropna().copy()

    veri = veri[
        veri[degisken].isin(["Var", "Yok"])
    ]

    capraz_tablo = pd.crosstab(
        veri["Grup"],
        veri[degisken],
    ).reindex(
        index=["Hasta", "Kontrol"],
        columns=["Var", "Yok"],
        fill_value=0,
    )

    if int(capraz_tablo.values.sum()) > 0:
        odds_ratio, p_degeri = fisher_exact(
            capraz_tablo.values,
            alternative="two-sided",
        )

        yorum = (
            "Significant association"
            if p_degeri < ANLAMLILIK_DUZEYI
            else "No significant association"
        )
    else:
        odds_ratio = np.nan
        p_degeri = np.nan
        yorum = "Test could not be calculated"

    kategorik_sonuclar.append({
        "Variable": degisken,
        "Patient_Present": int(
            capraz_tablo.loc["Hasta", "Var"]
        ),
        "Patient_Absent": int(
            capraz_tablo.loc["Hasta", "Yok"]
        ),
        "Control_Present": int(
            capraz_tablo.loc["Kontrol", "Var"]
        ),
        "Control_Absent": int(
            capraz_tablo.loc["Kontrol", "Yok"]
        ),
        "Odds_Ratio": odds_ratio,
        "p_Value": p_degeri,
        "Interpretation": yorum,
    })

kategorik_tablo = pd.DataFrame(
    kategorik_sonuclar
)
```

3.3 SPEARMAN CORRELATION

Spearman correlation examines monotonic relationships between DHI scores and objective clinical test parameters within the patient group.

```
vm_veri = hipotez_tablo[
    hipotez_tablo["Grup"] == "Hasta"
].copy()

korelasyon_sonuclari = []

for degisken in korelasyon_degiskenleri:
    alt_veri = vm_veri[
        ["DHI_Toplam", degisken]
    ].dropna()

    yeterli_veri = (
        len(alt_veri) >= 3
        and alt_veri["DHI_Toplam"].nunique() > 1
        and alt_veri[degisken].nunique() > 1
    )

    if yeterli_veri:
        rho, p_degeri = spearmanr(
            alt_veri["DHI_Toplam"],
            alt_veri[degisken],
        )

        yon = (
            "Negative"
            if rho < 0
            else "Positive"
            if rho > 0
            else "None"
        )

        yorum = (
            "Significant correlation"
            if p_degeri < ANLAMLILIK_DUZEYI
            else "No significant correlation"
        )
    else:
        rho = np.nan
        p_degeri = np.nan
        yon = "Not calculated"
        yorum = "Insufficient or constant data"

    korelasyon_sonuclari.append({
        "Variable": degisken,
        "n": len(alt_veri),
        "Spearman_rho": rho,
        "p_Value": p_degeri,
        "Direction": yon,
        "Interpretation": yorum,
    })

korelasyon_tablo = pd.DataFrame(
    korelasyon_sonuclari
)
```

4. AUTOMATED EXCEL & WORD REPORTING

This section exports structured statistical results to a multi-sheet Excel workbook and generates an academic-style Word report with formatted tables and interpretations.

4.1 MULTI-SHEET EXCEL OUTPUT

The workflow exports cleaned data, descriptive statistics, hypothesis tests, correlations, data-quality checks, and technical logs to separate Excel worksheets.

```
with pd.ExcelWriter(
    CIKTI_EXCEL_YOLU,
    engine="openpyxl",
) as writer:

    ana_kisi.to_excel(
        writer,
        sheet_name="Participants",
        index=False,
    )

    kullanilabilir_ozet.to_excel(
        writer,
        sheet_name="Available_Data",
        index=False,
    )

    tanimsal_tablo.to_excel(
        writer,
        sheet_name="Descriptive_Statistics",
        index=False,
    )

    istatistik_tablo.to_excel(
        writer,
        sheet_name="Mann_Whitney",
        index=False,
    )

    kategorik_tablo.to_excel(
        writer,
        sheet_name="Fisher_Exact",
        index=False,
    )

    korelasyon_tablo.to_excel(
        writer,
        sheet_name="Spearman_Correlation",
        index=False,
    )

    sutun_esleme_tablo.to_excel(
        writer,
        sheet_name="Column_Mapping",
        index=False,
    )

    uyarilar_tablo.to_excel(
        writer,
        sheet_name="Quality_Warnings",
        index=False,
    )

    print("Excel analysis output saved:", CIKTI_EXCEL_YOLU)
```

4.2 AUTOMATED WORD REPORT GENERATION

The workflow generates a structured Word report containing the study methodology, statistical results, formatted tables, and automated interpretations.

```
from docx import Document
from docx.shared import Pt

def tabloyu_worde_ekle(
    belge,
    veri_tablosu,
    baslik,
):
    belge.add_heading(baslik, level=2)

    tablo = belge.add_table(
        rows=1,
        cols=len(veri_tablosu.columns),
    )
    tablo.style = "Table Grid"

    for sutun_no, sutun_adi in enumerate(
        veri_tablosu.columns
    ):
        tablo.rows[0].cells[sutun_no].text = str(
            sutun_adi
        )

    for _, satir in veri_tablosu.iterrows():
        hucreler = tablo.add_row().cells

        for sutun_no, deger in enumerate(satir):
            hucreler[sutun_no].text = (
                ""
                if pd.isna(deger)
                else str(deger)
            )

belge = Document()

belge.add_heading(
    "Clinical Data Analysis Report",
    level=0,
)

belge.add_paragraph(
    "Anonymized 20-participant pilot study"
)

belge.add_heading("Methodology", level=1)
belge.add_paragraph(
    "Continuous variables were compared using "
    "the Mann-Whitney U test. Categorical findings "
    "were evaluated with Fisher's exact test, and "
    "relationships were examined using Spearman "
    "correlation analysis."
)

tabloyu_worde_ekle(
    belge,
    tanimsal_tablo,
    "Descriptive Statistics",
)
```

4.2 AUTOMATED WORD REPORT GENERATION— CONTINUED

```
tabloyu_worde_ekle(  
    belge,  
    istatistik_tablo,  
    "Mann-Whitney U Test",  
)  
  
tabloyu_worde_ekle(  
    belge,  
    kategorik_tablo,  
    "Fisher's Exact Test",  
)  
  
tabloyu_worde_ekle(  
    belge,  
    korelasyon_tablo,  
    "Spearman Correlation",  
)  
  
belge.save(CIKTI_WORD_YOLU)  
print("Word report saved:", CIKTI_WORD_YOLU)
```

5. WORKFLOW SUMMARY & DELIVERABLES

This portfolio sample demonstrates an end-to-end clinical data analysis workflow, from privacy-safe Excel processing and statistical testing to automated Excel and Word report generation.

KEY DELIVERABLES

- Cleaned and standardized clinical dataset
- Descriptive and non-parametric statistical analyses
- Structured multi-sheet Excel analysis output
- Automated academic-style Word report